קטע קוד A

```
#include <iostream>
using namespace std;

template<class T>
class Inner
{
	T t;
public:
	Inner(const T& t) : t(t)
	{
		cout << "creating Inner t=" << t << endl;
	}

	Inner(const Inner& other) : t(other.t)
	{
		cout << "copying Inner t=" << t << endl;
	}

	~Inner()
	{
		cout << "destryoing Inner t=" << t << endl;
	}

	const T& getT() const { return t; }

	operator T() const { return t; }
};

template<class T>
class Base
{
	static int counter;
	Inner<T> i;
public:
	Base(const Inner<T>& i) : i(i)
	{
		counter++;
		cout << "creating Base with Inner type |" <<
		    typeid(i.getT()).name() << "| = " << i.getT() << endl;
	}

	~Base()
	{
		cout << "~Base with Inner t = " << i.getT()
			<< ", counter=" << counter-- << endl;
	}

	friend ostream& operator<<(ostream& os, const Base& b)
	{
		os << b.i.getT() << endl;
		return os;
	}
};
```

```
template<class T>
int Base<T>::counter = 0;

template<class T1, class T2>
class Derived : public Base<T1>
{
	T2 t2;
public:
	Derived(const T1& t1, const T2& t2) : Base(t1), t2(t2)
	{
		cout << "creating Derived with |" << typeid(t2).name()
			<< "| = " << t2 << endl;
	}
 };
```

להלן מספר main אפשריים:

A main

```
int main()
{
	Base<int> b1(5);
	cout << "-------------\n";
	Base<int> b2(b1);
	cout << "-------------\n";
	return 0;
 }
```

B main

```
int main()
{
	Derived<int, char> d1(6, 'a');
	cout << "-------------\n";
	return 0;
 }
```

C main

```
int main()
{
	Derived<Inner<int>, Inner<char>> d(6, 'a');
	return 0;
 }
```

D main

```
int main()
{
	Inner<int> i(6);
	{
		Derived<Base<int>, char> d(i, 'a');
		cout << "-------------\n";
	}
	cout << "-------------\n";
	return 0;
 }
```

קטע קוד B

```
#include <iostream>
using namespace std;

class A
{
	int x;
public:
	A(int x = 8) : x(x) { cout << "A x=" << x << endl; }

	virtual void foo() const { cout << "A::foo\n"; }
		 void goo() const { cout << "A::goo\n"; }

	virtual ~A() { cout << "~A\n"; }

	void toOs(ostream& os) const {}

	friend ostream& operator<<(ostream& os, const A& a)
	{
		os << "printing A\n";
		a.toOs(os);
		return os;
	}
};

class B : public A
{
public:
	virtual void foo()       { cout << "B::foo\n"; }
	virtual void goo() const { cout << "B::goo\n"; }

	virtual void toOs(ostream& os) const { os << "B::toOs\n"; }

	~B() { cout << "~B\n"; }
};

class C : public B
{
public:
	virtual void foo() const { cout << "C::foo\n"; }
	virtual void goo() const { cout << "C::goo\n"; }

	virtual void toOs(ostream& os) const { os << "C::toOs\n"; }

	~C() { cout << "~C\n"; }

	friend ostream& operator<<(ostream& os, const C& c)
	{
		os << "printing C\n";
		return os;
	}

	static void moo(B param)
	{
		cout << "C::moo param type is " << (typeid(param).name() + 6)
		     << endl;
	}
};
```

```
class D
{
public:
	D() { cout << "D\n"; }
	~D() { cout << "~D\n"; }
	operator C() const { cout << "D::operator C\n"; return C(); }
};
```

להלן מספר main אפשריים:

A main

```
int main()
{
	C::moo(D());
	cout << "-----------\n";
	return 0;
}
```

B main

```
int main()
{
	A* a2 = new C();
	C* c = dynamic_cast<C*>(a2);
	if (c)
	    c->moo(*c);
	delete a2;

	return 0;
}
```

C main

```
int main()
{
	C c;
	B& b = c;
	b.foo();
	b.goo();
	cout << "--------------\n";
	cout << b;

	return 0;
 }
```

D main

```
int main ()
{
	D d;
	((C)d).foo();
	((C)d).goo();

	cout << (C)d << endl;

	return 0;
 }
```

קטע קוד C

```
#include <iostream>
using namespace std;

class Stam
{
	char* s;
public:
	Stam(const char* s)
	{
		this->s = new char[strlen(s)+1];
		strcpy(this->s, s);
	}

	Stam(const Stam& other) : s(nullptr)
	{
		*this = other;
	}

	Stam(Stam&& other)
	{
		this->s = other.s;
		other.s = nullptr;
	}

	const Stam& operator=(const Stam& other)
	{
		if (this != &other)
		{
			delete[]s;
			this->s = new char[strlen(other.s) + 1];
			strcpy(this->s, other.s);
		}
		return *this;
	}

	Stam& operator=(Stam&& other)
	{
		if (this != &other)
		{
			delete[]s;
			this->s = other.s;
			other.s = nullptr;
		}
		return *this;
	}

	~Stam()
	{
		if (s)
			cout << "deleting " << s << endl;
		delete[]s;
	}

	operator const char*() { return s; }
};

Stam foo()
{
	Stam s("kuku");
	return s;
}
```

```
int main()
{
	Stam s1("hi");
	Stam s2("bye");

	Stam s3(s1);
	s1 = foo();

	return 0;
}
```

בהצלחה !